1. Environment Setup & Library ImportsWe configure Keras (compatible with PyTorch or TensorFlow backends) and initialize random seeds for classroom reproducibility.

```
# Set Keras backend to 'torch' if TensorFlow is not available, or default
import os
if "KERAS_BACKEND" not in os.environ:
    os.environ["KERAS_BACKEND"] = "torch"

# Core Data Science Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-Learn for Preprocessing & Evaluation
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, classification_report
)

# Deep Learning with Keras
import keras
from keras import layers, models, callbacks

# Set random seeds for reproducibility
np.random.seed(42)
keras.utils.set_random_seed(42)

# Visualization settings
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['font.size'] = 11

print("=" * 60)
print(f"Keras Version : {keras.__version__}")
print(f"Backend Engine: {keras.backend.backend()}")
print(f"Pandas Version: {pd.__version__}")
print(f"NumPy Version : {np.__version__}")
print("=" * 60)
```

```
============================================================
Keras Version : 3.13.2
Backend Engine: torch
Pandas Version: 2.2.3
NumPy Version : 2.1.3
============================================================
```

```
df=pd.read_csv('airline_passenger_satisfaction.csv')
df.head()
```

	Unnamed: 0	id	Gender	Customer Type	Age	Type of Travel	Class	Flight Distance	Inflight wifi service	Departure/Arrival time convenient	...	Inflight entertainment	On-board service
0	0	70172	Male	Loyal Customer	13	Personal Travel	Eco Plus	460	3	4	...	5	4
1	1	5047	Male	disloyal Customer	25	Business travel	Business	235	3	2	...	1	1
2	2	110028	Female	Loyal Customer	26	Business travel	Business	1142	2	2	...	5	4
3	3	24026	Female	Loyal Customer	25	Business travel	Business	562	2	5	...	2	2
4	4	119299	Male	Loyal Customer	61	Business travel	Business	214	3	3	...	3	3

5 rows × 25 columns

2.1 Inspecting Column Names and Data Types

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103904 entries, 0 to 103903
Data columns (total 25 columns):
 #   Column                            Non-Null Count   Dtype  
---  ------                            --------------   -----  
 0   Unnamed: 0                        103904 non-null  int64  
 1   id                                103904 non-null  int64  
 2   Gender                            103904 non-null  object 
 3   Customer Type                     103904 non-null  object 
 4   Age                               103904 non-null  int64  
 5   Type of Travel                    103904 non-null  object 
 6   Class                             103904 non-null  object 
 7   Flight Distance                   103904 non-null  int64  
 8   Inflight wifi service             103904 non-null  int64  
 9   Departure/Arrival time convenient  103904 non-null  int64  
 10  Ease of Online booking            103904 non-null  int64  
 11  Gate location                     103904 non-null  int64  
 12  Food and drink                    103904 non-null  int64  
 13  Online boarding                   103904 non-null  int64  
 14  Seat comfort                      103904 non-null  int64  
 15  Inflight entertainment            103904 non-null  int64  
 16  On-board service                  103904 non-null  int64  
 17  Leg room service                  103904 non-null  int64  
 18  Baggage handling                  103904 non-null  int64  
 19  Checkin service                   103904 non-null  int64  
 20  Inflight service                  103904 non-null  int64  
 21  Cleanliness                       103904 non-null  int64  
 22  Departure Delay in Minutes        103904 non-null  int64  
 23  Arrival Delay in Minutes          103594 non-null  float64
 24  satisfaction                      103904 non-null  object 
dtypes: float64(1), int64(19), object(5)
memory usage: 19.8+ MB
```

```
# Summary statistics of numerical columns
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Unnamed: 0	103904.0	51951.500000	29994.645522	0.0	25975.75	51951.5	77927.25	103903.0
id	103904.0	64924.210502	37463.812252	1.0	32533.75	64856.5	97368.25	129880.0
Age	103904.0	39.379706	15.114964	7.0	27.00	40.0	51.00	85.0
Flight Distance	103904.0	1189.448375	997.147281	31.0	414.00	843.0	1743.00	4983.0
Inflight wifi service	103904.0	2.729683	1.327829	0.0	2.00	3.0	4.00	5.0
Departure/Arrival time convenient	103904.0	3.060296	1.525075	0.0	2.00	3.0	4.00	5.0
Ease of Online booking	103904.0	2.756901	1.398929	0.0	2.00	3.0	4.00	5.0
Gate location	103904.0	2.976883	1.277621	0.0	2.00	3.0	4.00	5.0
Food and drink	103904.0	3.202129	1.329533	0.0	2.00	3.0	4.00	5.0
Online boarding	103904.0	3.250375	1.349509	0.0	2.00	3.0	4.00	5.0
Seat comfort	103904.0	3.439396	1.319088	0.0	2.00	4.0	5.00	5.0
Inflight entertainment	103904.0	3.358158	1.332991	0.0	2.00	4.0	4.00	5.0
On-board service	103904.0	3.382363	1.288354	0.0	2.00	4.0	4.00	5.0
Leg room service	103904.0	3.351055	1.315605	0.0	2.00	4.0	4.00	5.0
Baggage handling	103904.0	3.631833	1.180903	1.0	3.00	4.0	5.00	5.0
Checkin service	103904.0	3.304290	1.265396	0.0	3.00	3.0	4.00	5.0
Inflight service	103904.0	3.640428	1.175663	0.0	3.00	4.0	5.00	5.0
Cleanliness	103904.0	3.286351	1.312273	0.0	2.00	3.0	4.00	5.0
Departure Delay in Minutes	103904.0	14.815618	38.230901	0.0	0.00	0.0	12.00	1592.0
Arrival Delay in Minutes	103594.0	15.178678	38.698682	0.0	0.00	0.0	13.00	1584.0

```
# Check missing values
missing_counts = df.isnull().sum()
missing_cols = missing_counts[missing_counts > 0]
print("Columns with missing values:")
print(missing_cols)
print(f"\nMissing percentage for Arrival Delay: {(missing_cols.iloc[0] / len(df)) * 100:.2f}%")
```

```
Columns with missing values:
Arrival Delay in Minutes    310
dtype: int64

Missing percentage for Arrival Delay: 0.30%
```

```
# 1. Drop redundant columns
cols_to_drop = [c for c in ['Unnamed: 0', 'id'] if c in df.columns]
df = df.drop(columns=cols_to_drop)
print(f"Dropped non-predictive columns: {cols_to_drop}")

# 2. Impute missing Arrival Delays with median
median_arrival_delay = df['Arrival Delay in Minutes'].median()
df['Arrival Delay in Minutes'] = df['Arrival Delay in Minutes'].fillna(median_arrival_delay)
print(f"Imputed missing Arrival Delay with median value: {median_arrival_delay} minutes")

# 3. Verification
print(f"Remaining null values across dataset: {df.isnull().sum().sum()}")
print(f"Cleaned dataset shape: {df.shape[0]:,} rows × {df.shape[1]} columns")
```

```
Dropped non-predictive columns: []
Imputed missing Arrival Delay with median value: 0.0 minutes
Remaining null values across dataset: 0
Cleaned dataset shape: 103,904 rows × 23 columns
```

```
sat_counts = df['satisfaction'].value_counts()
sat_pcts = df['satisfaction'].value_counts(normalize=True) * 100

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Bar Chart
sns.barplot(x=sat_counts.index, y=sat_counts.values, palette=['#d95f02', '#2b5c8f'], ax=axes[0])
axes[0].set_title(f"Passenger Satisfaction Count (Total: {len(df):,})", fontsize=14, fontweight='bold')
axes[0].set_ylabel("Number of Passengers")
for i, v in enumerate(sat_counts.values):
    axes[0].text(i, v + 1200, f"{v:,} ({sat_pcts.iloc[i]:.1f}%)", ha='center', fontweight='bold')

# Donut Chart
axes[1].pie(sat_counts, labels=sat_counts.index, autopct='%1.1f%%',
            colors=['#d95f02', '#2b5c8f'], startangle=90, explode=(0, 0.05),
            wedgeprops=dict(width=0.4, edgecolor='w'))
axes[1].set_title("Satisfaction Proportion", fontsize=14, fontweight='bold')

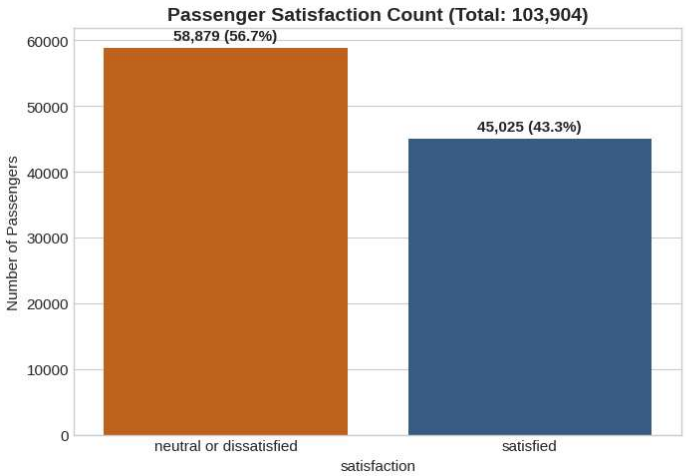
plt.tight_layout()
plt.show()

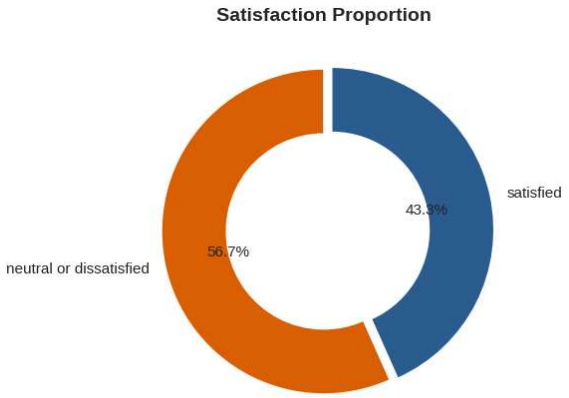
print(f"Neutral or Dissatisfied (0): {sat_counts['neutral or dissatisfied']:,} ({sat_pcts['neutral or dissatisfied']:.2f}%)")
print(f"Satisfied (1)              : {sat_counts['satisfied']:,} ({sat_pcts['satisfied']:.2f}%)")
```

```
/tmp/ipykernel_910/2290261522.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  sns.barplot(x=sat_counts.index, y=sat_counts.values, palette=['#d95f02', '#2b5c8f'], ax=axes[0])
```






```
Neutral or Dissatisfied (0): 58,879 (56.67%)
Satisfied (1)              : 45,025 (43.33%)
```

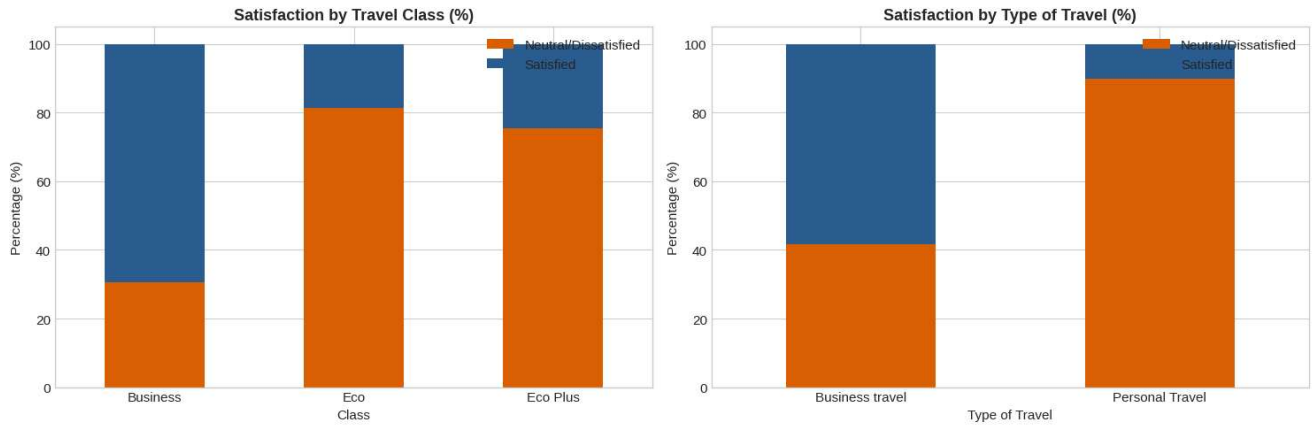
4.2 Impact of Flight Class and Travel Type

```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Class vs Satisfaction
class_sat = pd.crosstab(df['Class'], df['satisfaction'], normalize='index') * 100
class_sat.plot(kind='bar', stacked=True, color=['#d95f02', '#2b5c8f'], ax=axes[0], rot=0)
axes[0].set_title("Satisfaction by Travel Class (%)", fontsize=13, fontweight='bold')
axes[0].set_ylabel("Percentage (%)")
axes[0].legend(["Neutral/Dissatisfied", "Satisfied"], loc="upper right")

# Type of Travel vs Satisfaction
travel_sat = pd.crosstab(df['Type of Travel'], df['satisfaction'], normalize='index') * 100
travel_sat.plot(kind='bar', stacked=True, color=['#d95f02', '#2b5c8f'], ax=axes[1], rot=0)
axes[1].set_title("Satisfaction by Type of Travel (%)", fontsize=13, fontweight='bold')
axes[1].set_ylabel("Percentage (%)")
axes[1].legend(["Neutral/Dissatisfied", "Satisfied"], loc="upper right")

plt.tight_layout()
plt.show()
```



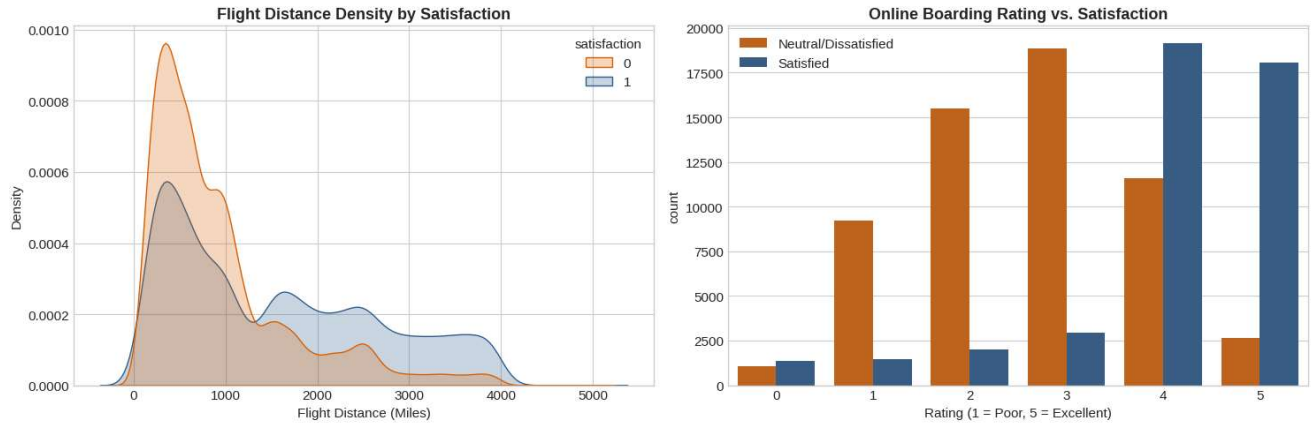
4.3 Flight Distance and Inflight Service Ratings

```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Flight Distance Density by Satisfaction
sns.kdeplot(data=df, x='Flight Distance', hue='satisfaction', common_norm=False, fill=True,
            palette=['#d95f02', '#2b5c8f'], ax=axes[0])
axes[0].set_title("Flight Distance Density by Satisfaction", fontsize=13, fontweight='bold')
axes[0].set_xlabel("Flight Distance (Miles)")

# Online Boarding Rating vs Satisfaction
sns.countplot(data=df, x='Online boarding', hue='satisfaction',
              palette=['#d95f02', '#2b5c8f'], ax=axes[1])
axes[1].set_title("Online Boarding Rating vs. Satisfaction", fontsize=13, fontweight='bold')
axes[1].set_xlabel("Rating (1 = Poor, 5 = Excellent)")
axes[1].legend(["Neutral/Dissatisfied", "Satisfied"], loc="upper left")

plt.tight_layout()
plt.show()
```




```
# 1. Encode Target
# Check if 'satisfaction' column is still of object type before processing
if df['satisfaction'].dtype == 'object':
    # Clean the 'satisfaction' column by stripping whitespace before mapping to prevent NaN values.
```

```
    df['satisfaction'] = df['satisfaction'].str.strip().map({'satisfied': 1, 'neutral or dissatisfied': 0}).astype(int)

# 2. Identify categorical feature columns
# Ensure 'satisfaction' is not included if it's already numeric
cat_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()
print(f"Categorical features to encode ({len(cat_cols)}): {cat_cols}")

# 3. One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True, dtype=float)

print(f"Encoded dataset dimensions: {df_encoded.shape[0]:,} rows × {df_encoded.shape[1]} columns")
df_encoded.head()
```

```
Categorical features to encode (4): ['Gender', 'Customer Type', 'Type of Travel', 'Class']
Encoded dataset dimensions: 103,904 rows × 24 columns
```

	Age	Flight Distance	Inflight wifi service	Departure/Arrival time convenient	Ease of Online booking	Gate location	Food and drink	Online boarding	Seat comfort	Inflight entertainment	...	Inflight service	Cleanli
0	13	460	3	4	3	1	5	3	5	5	...	5	
1	25	235	3	2	3	3	1	3	1	1	...	4	
2	26	1142	2	2	2	2	5	5	5	5	...	4	
3	25	562	2	5	5	5	2	2	2	2	...	4	
4	61	214	3	3	3	3	4	5	5	3	...	3	

5 rows × 24 columns

```
# Separate features and target
X = df_encoded.drop(columns=['satisfaction'])
y = df_encoded['satisfaction'].values

# Split with stratification
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print(f"Training set : {X_train.shape[0]:,} samples, {X_train.shape[1]} features")
print(f"Testing set  : {X_test.shape[0]:,} samples, {X_test.shape[1]} features")
print(f"Train Class 1 ratio: {np.mean(y_train):.3f}")
print(f"Test Class 1 ratio : {np.mean(y_test):.3f}")
```

```
Training set : 83,123 samples, 23 features
Testing set  : 20,781 samples, 23 features
Train Class 1 ratio: 0.433
Test Class 1 ratio : 0.433
```

```
scaler = StandardScaler()

# Fit on train, transform both
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Compare before and after
scale_comparison = pd.DataFrame({
    'Raw Train Mean': X_train.mean()[:5],
    'Raw Train Std': X_train.std()[:5],
    'Scaled Train Mean': X_train_scaled.mean(axis=0)[:5].round(4),
    'Scaled Train Std': X_train_scaled.std(axis=0)[:5].round(4)
})
display(scale_comparison)
```

	Raw Train Mean	Raw Train Std	Scaled Train Mean	Scaled Train Std
Age	39.393994	15.127582	0.0	1.0
Flight Distance	1186.755290	995.226119	0.0	1.0
Inflight wifi service	2.731518	1.327142	0.0	1.0
Departure/Arrival time convenient	3.060765	1.526677	0.0	1.0
Ease of Online booking	2.758057	1.398677	0.0	1.0

```
num_features = X_train_scaled.shape[1]

# Define Model Architecture
model = models.Sequential([
    layers.Input(shape=(num_features,)),

    # Hidden Layer 1
    layers.Dense(64, activation='relu', name='dense_1'),
    layers.BatchNormalization(name='batch_norm_1'),
    layers.Dropout(0.20, name='dropout_1'),

    # Hidden Layer 2
    layers.Dense(32, activation='relu', name='dense_2'),
    layers.Dropout(0.10, name='dropout_2'),

    # Hidden Layer 3
    layers.Dense(16, activation='relu', name='dense_3'),

    # Output Layer
    layers.Dense(1, activation='sigmoid', name='output_layer')
])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 64)	1,536
batch_norm_1 (BatchNormalization)	(None, 64)	256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dropout_2 (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 16)	528
output_layer (Dense)	(None, 1)	17

Total params: 4,417 (17.25 KB)

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

early_stopping = callbacks.EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True,
    verbose=1
)

EPOCHS = 15
BATCH_SIZE = 128

print("Starting Model Training on 83,123 records...")
history = model.fit(
    X_train_scaled, y_train,
    validation_split=0.20,  # 20% of training data reserved for validation
    epochs=EPOCHS,
```

```
    batch_size=BATCH_SIZE,
    callbacks=[early_stopping],
    verbose=1
)
print("Training Complete!")
```

```
Starting Model Training on 83,123 records...
Epoch 1/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9540 - loss: 0.1091 - val_accuracy: 0.9578 - val_loss: 0.1011
Epoch 2/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.9557 - loss: 0.1063 - val_accuracy: 0.9572 - val_loss: 0.1035
Epoch 3/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 18ms/step - accuracy: 0.9553 - loss: 0.1059 - val_accuracy: 0.9561 - val_loss: 0.1017
Epoch 4/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9559 - loss: 0.1042 - val_accuracy: 0.9571 - val_loss: 0.1006
Epoch 5/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9558 - loss: 0.1042 - val_accuracy: 0.9584 - val_loss: 0.0992
Epoch 6/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9564 - loss: 0.1026 - val_accuracy: 0.9584 - val_loss: 0.0994
Epoch 7/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 18ms/step - accuracy: 0.9572 - loss: 0.1018 - val_accuracy: 0.9579 - val_loss: 0.0987
Epoch 8/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9572 - loss: 0.1018 - val_accuracy: 0.9577 - val_loss: 0.0991
Epoch 9/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9577 - loss: 0.1002 - val_accuracy: 0.9577 - val_loss: 0.1005
Epoch 10/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 9s 18ms/step - accuracy: 0.9573 - loss: 0.0995 - val_accuracy: 0.9575 - val_loss: 0.0985
Epoch 11/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9581 - loss: 0.0985 - val_accuracy: 0.9586 - val_loss: 0.0972
Epoch 12/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9576 - loss: 0.0999 - val_accuracy: 0.9585 - val_loss: 0.0994
Epoch 13/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9583 - loss: 0.0980 - val_accuracy: 0.9568 - val_loss: 0.1008
Epoch 14/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 9s 17ms/step - accuracy: 0.9592 - loss: 0.0971 - val_accuracy: 0.9600 - val_loss: 0.0966
Epoch 15/15
520/520 ━━━━━━━━━━━━━━━━━━━━ 10s 19ms/step - accuracy: 0.9588 - loss: 0.0970 - val_accuracy: 0.9603 - val_loss: 0.0963
Restoring model weights from the end of the best epoch: 15.
Training Complete!
```

10. Visualizing Training & Validation Performance

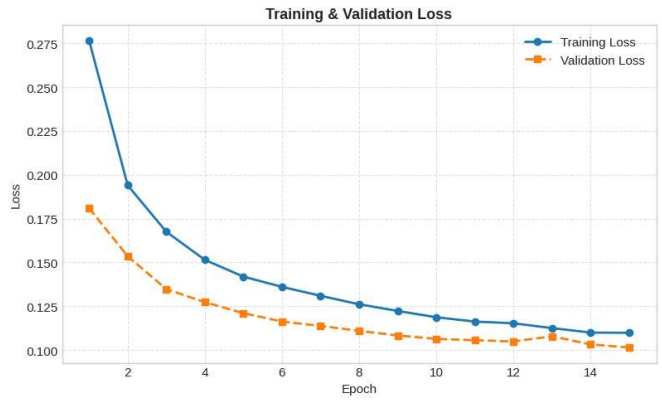
```
history_dict = history.history
epochs_range = range(1, len(history_dict['loss']) + 1)

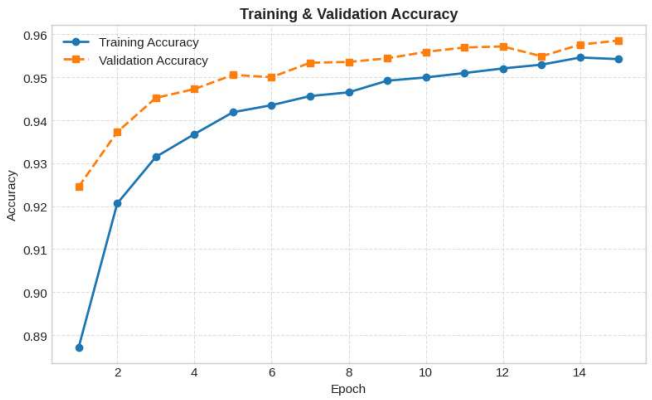
fig, axes = plt.subplots(1, 2, figsize=(16, 5))

# Loss Curve
axes[0].plot(epochs_range, history_dict['loss'], 'o-', label='Training Loss', color='#1f77b4', linewidth=2)
axes[0].plot(epochs_range, history_dict['val_loss'], 's--', label='Validation Loss', color='#ff7f0e', linewidth=2)
axes[0].set_title('Training & Validation Loss', fontsize=13, fontweight='bold')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].legend()
axes[0].grid(True, linestyle='--', alpha=0.6)

# Accuracy Curve
axes[1].plot(epochs_range, history_dict['accuracy'], 'o-', label='Training Accuracy', color='#1f77b4', linewidth=2)
axes[1].plot(epochs_range, history_dict['val_accuracy'], 's--', label='Validation Accuracy', color='#ff7f0e', linewidth=2)
axes[1].set_title('Training & Validation Accuracy', fontsize=13, fontweight='bold')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()
```






```
# Predict probabilities
y_pred_probs = model.predict(X_test_scaled, verbose=0).ravel()

# Binary predictions using 0.5 threshold
y_pred = (y_pred_probs >= 0.5).astype(int)

test_acc = accuracy_score(y_test, y_pred)
test_prec = precision_score(y_test, y_pred)
test_rec = recall_score(y_test, y_pred)
test_f1 = f1_score(y_test, y_pred)
test_auc = roc_auc_score(y_test, y_pred_probs)

print("=" * 60)
print(f"TEST SET PERFORMANCE METRICS (N = {len(y_test):,} Passengers)")
print("=" * 60)
print(f"Accuracy  : {test_acc * 100:.2f}%")
print(f"Precision : {test_prec * 100:.2f}%")
print(f"Recall    : {test_rec * 100:.2f}%")
print(f"F1-Score  : {test_f1 * 100:.2f}%")
print(f"ROC-AUC   : {test_auc:.4f}")
print("=" * 60)
```

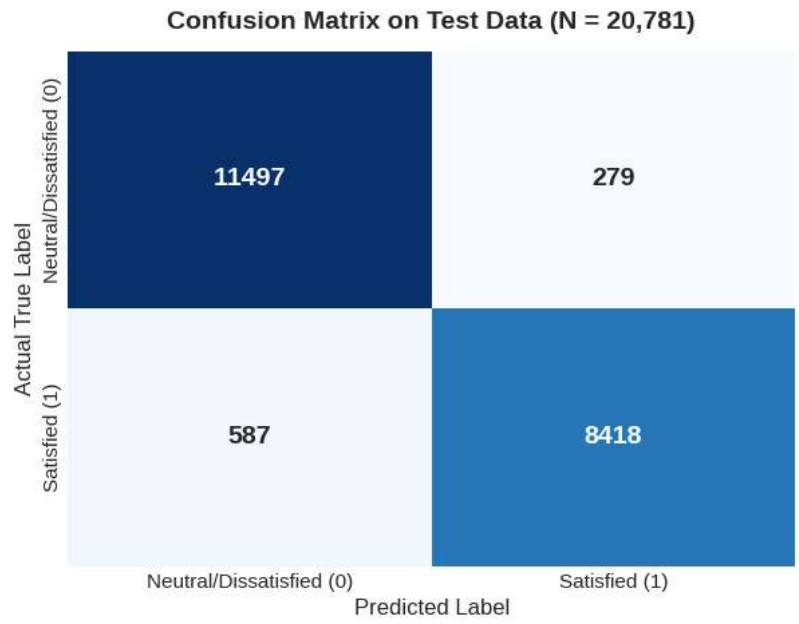
```
============================================================
TEST SET PERFORMANCE METRICS (N = 20,781 Passengers)
============================================================
Accuracy  : 95.83%
Precision : 96.79%
Recall    : 93.48%
F1-Score  : 95.11%
ROC-AUC   : 0.9936
============================================================
```

11.1 Confusion Matrix Analysis

```
cm = confusion_matrix(y_test, y_pred)
cm_labels = ['Neutral/Dissatisfied (0)', 'Satisfied (1)']

plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=cm_labels, yticklabels=cm_labels,
            cbar=False, annot_kws={'size': 14, 'weight': 'bold'})
plt.title(f'Confusion Matrix on Test Data (N = {len(y_test):,})', fontsize=14, fontweight='bold', pad=12)
plt.xlabel('Predicted Label', fontsize=12)
plt.ylabel('Actual True Label', fontsize=12)
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f"True Negatives  : {tn:,}")
print(f"False Positives : {fp:,}")
print(f"False Negatives : {fn:,}")
print(f"True Positives  : {tp:,}")
```




```
True Negatives  : 11,497
False Positives : 279
False Negatives : 587
True Positives  : 8,418
```

11.2 Classification Report & ROC Curve

```
print("Detailed Classification Report:")
print(classification_report(y_test, y_pred, target_names=['Neutral/Dissatisfied', 'Satisfied']))

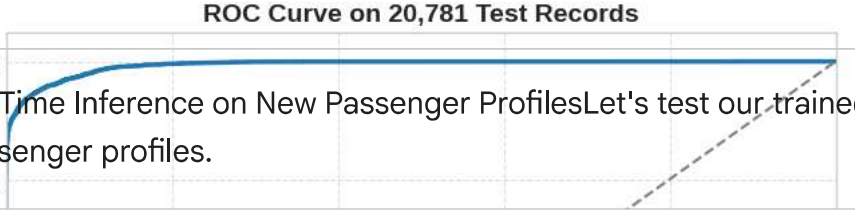
fpr, tpr, _ = roc_curve(y_test, y_pred_probs)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='#1f77b4', lw=2.5, label=f'ANN ROC Curve (AUC = {test_auc:.4f})')
plt.plot([0, 1], [0, 1], color='gray', lw=1.5, linestyle='--', label='Random Guess (AUC = 0.500)')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate (Recall)', fontsize=12)
plt.title(f'ROC Curve on 20,781 Test Records', fontsize=14, fontweight='bold')
plt.legend(loc='lower right', fontsize=11)
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

```
Detailed Classification Report:
                     precision    recall  f1-score   support

Neutral/Dissatisfied       0.95      0.98      0.96     11776
           Satisfied       0.97      0.93      0.95      9005

            accuracy                           0.96     20781
           macro avg       0.96      0.96      0.96     20781
        weighted avg       0.96      0.96      0.96     20781
```



12. Real-Time Inference on New Passenger ProfilesLet's test our trained model on two realistic, new passenger profiles.

```
new_passengers_raw = [
    {
        # Profile 1: Discontented Eco Traveler
        'Gender': 'Female',
        'Customer Type': 'Loyal Customer',
        'Age': 32,
        'Type of Travel': 'Personal Travel',
        'Class': 'Eco',
        'Flight Distance': 1200,
        'Inflight wifi service': 1,
        'Departure/Arrival time convenient': 2,
        'Ease of Online booking': 1,
        'Gate location': 3,
        'Food and drink': 1,
        'Online boarding': 2,
        'Seat comfort': 1,
        'Inflight entertainment': 1,
        'On-board service': 2,
        'Leg room service': 1,
        'Baggage handling': 2,
        'Checkin service': 2,
        'Inflight service': 2,
        'Cleanliness': 1,
        'Departure Delay in Minutes': 75,
        'Arrival Delay in Minutes': 82.0
    },
    {
        # Profile 2: Delighted Business Traveler
        'Gender': 'Male',
        'Customer Type': 'Loyal Customer',
        'Age': 45,
        'Type of Travel': 'Business travel',
        'Class': 'Business',
        'Flight Distance': 2500,
        'Inflight wifi service': 5,
        'Departure/Arrival time convenient': 4,
        'Ease of Online booking': 5,
        'Gate location': 4,
        'Food and drink': 5,
        'Online boarding': 5,
        'Seat comfort': 5,
        'Inflight entertainment': 5,
        'On-board service': 5,
        'Leg room service': 5,
        'Baggage handling': 5,
        'Checkin service': 5,
```